Real-Time Ocean Rendering in Unreal Engine 5

A Hand-Built GPU FFT Wave Simulation and Measurement-Based Water Colour Model

Abhijit Shidlagatta

September 30, 2026

Technical write-up for the `WaterGraphicsV2` ocean system.
The wave simulation, the water colour model and the runtime preset system described here are built directly on Unreal Engine's compute shaders, its Render Dependency Graph and its Single Layer Water shading model without using Epic's Water plugin, `WaterBodyOcean` actor, or Gerstner wave components.

Contents

Abstract **4**

1 Introduction **5**
1.1 Why not just use Unreal's Water plugin . . . 5
1.2 What water actually has to get right . . . 5
1.3 How this report is organised . . . 6

2 Output Summary **7**

3 Context **8**
3.1 Wave models in real-time graphics . . . 8
3.2 Water colour in real-time graphics . . . 8
3.3 Water optics . . . 9
3.4 Where this project sits . . . 9

4 Method **10**
4.1 Architecture . . . 10
4.2 Iteration 1: rejecting a hand-authored sum of waves . . . 10
4.3 Iteration 2: the Phillips spectrum . . . 11
4.4 Iteration 3: time evolution and choppy displacement . . . 12
4.5 Iteration 4: the FFT itself . . . 12
4.6 Iteration 5: inversion and multi-cascade compositing . . . 13
4.7 Iteration 6: a physically grounded water colour . . . 13
4.8 Iteration 7: Jerlov presets and the biome blend . . . 14
4.9 Iteration 8: from hand-wiring to a single function call . . . 15

5 Results **17**
5.1 Cost of the simulation . . . 17
5.2 Correctness testing . . . 17
5.3 Comparison against the alternatives . . . 17
5.4 Discussion . . . 17

6 Conclusions and Discussion **19**
6.1 Objectives review . . . 19
6.2 Limitations . . . 19
6.3 Future work . . . 19
6.4 Licensing and reuse . . . 20
6.5 Closing . . . 20

References **21**

A Code Listings **22**
A.1 The Phillips spectrum (Section 4.3) 22
A.2 Time evolution and choppy displacement (Section 4.4) 22
A.3 The FFT butterfly (Section 4, Iteration 4) 23
A.4 Multi-cascade orchestration (Section 4.6) 24
A.5 The preset system (Section 4.9) 24

Declaration of AI Use **26**

List of Figures

4.1 Data flow from CPU precompute through GPU simulation to the shaded pixel. The butterfly and noise lookup tables (top-left) are the only CPU-side cost; everything else happens on the GPU every frame via the Render Dependency Graph. 11
4.2 One radix-2 Cooley-Tukey butterfly. Two complex inputs A, B combine into two outputs using a single complex multiply and an add/subtract. `FFTButterflyCS` performs exactly this, once per grid cell, per stage, per axis. 12
4.3 The base water-colour wiring, isolated from the rest of the material graph: three parameter nodes feeding directly into the Single Layer Water output node. Section 4.8 extends this with a blend chain in front of each pin. 14
4.4 Per-channel absorption coefficients across the three presets. Ocean shows the steep red$\gg$green$>$blue drop-off of clear water; Murky is flatter and elevated across all channels, matching turbid water's much shorter visibility depth. 15
4.5 The two-stage blend ramp: Murky $\rightarrow$ Tropical over $x \in [6000, 10000]$ cm, then Tropical $\rightarrow$ Ocean over $x \in [16000, 20000]$ cm. Each preset's optical coefficients are linearly interpolated by these weights before reaching the Single Layer Water output node. 16

Abstract

Real-time water has to do two things at once: move like water, and look like water under changing light. Most small projects fake both: waves are a couple of panning normal maps, and colour is an artist-picked gradient with a Fresnel term over the top. This project attempts to solve both. The waves come from a hand-written GPU implementation of Jerry Tessendorf's FFT ocean method, running as compute shaders through Unreal's Render Dependency Graph, the same published technique behind *Sea of Thieves*. The colour starts from real water-optics data instead of a picked gradient: the open-ocean preset's absorption comes from Pope and Fry's measured pure-water absorption spectrum, and the other presets are tuned by hand to follow Jerlov's water-type classification, all feeding directly into Unreal's Single Layer Water shading model. Both systems sit beneath one function call, `ApplyWaterPreset`, which switches a whole water body between *Ocean*, *Tropical* and *Murky* looks at runtime with no material editor work at all. This report explains why each technique was chosen over the alternatives, walks through how it was built, and evaluates the result against Epic's own built-in Water plugin and the shipped-game techniques it draws on.

Chapter 1

Introduction

Water in a game has to do two things at once: move like water, and look like water under changing light. Getting either wrong results in a broken illusion: a well-lit sea with a flat, glassy surface looks fake, and a well-animated surface under a hand-picked blue gradient reads as fake too, just for a different reason.

Most small projects only solve half of this. Waves are usually two or three panning normal maps scrolling in different directions. Colour is usually a gradient picked in the material editor, with a Fresnel term (a reflectivity curve that gets stronger the closer you look at the water side-on) added so the horizon looks a bit mirror-like. They are both cheap and look fine from a distance but neither is simulating anything. They have the *look* of water, but not the physics of it. This project tries to simulate both instead.

1.1 Why not just use Unreal's Water plugin

Unreal already ships a complete ocean and river system: the Water plugin, `WaterBodyOcean` and `WaterBodyRiver` actors, built-in Gerstner wave actors, and collision-based submersion detection. For a team shipping a game, using it is the obviously correct call since it is free, tested and fast. I did not use it here for a different reason: the point of this project was to showcase that building a realistic approach to water rendering is possible. Every system in this report: the compute shader dispatch, the wave maths, the colour model, the material wiring was built from Unreal's lower-level primitives (compute shaders, `FGlobalShader`, the Render Dependency Graph, the Single Layer Water shading model) rather than assembled from a pre-built solution.

That is not a rejection of engine tooling in general. Lumen for lighting, the Sky Atmosphere system for sky colour, and Single Layer Water as a shading *model* (as opposed to a shading *solution*) are all still doing real work underneath this project. The line I am drawing is between *using a pre-built solution* and *building a solution from primitives*.

1.2 What water actually has to get right

Realistic water has three requirements that are each easy on their own and hard together:

1. **Correct wave shape.** Real waves are not one sine wave: they are thousands of overlapping frequencies and directions, sharper at the crest than the trough, and their statistics change with wind speed.
2. **Correct light behaviour.** Water colour comes from light being absorbed and scattered by wavelength as it passes through a volume, which is the reason the same body of water

can look deep blue or muddy brown depending on what is suspended in it.

3. **Real-time performance.** All of the above has to run every frame, at a size large enough to cover a playable level, without eating the whole frame budget.

Chapter 3 looks at how existing games and the graphics literature solve these three problems. Chapter 4 is the bulk of this report: how I solved them, in the order I built the solution.

1.3 How this report is organised

Chapter 2 lists what this project actually delivers. Chapter 3 covers the existing techniques and the published research this project builds on. Chapter 4 walks through the implementation, iteration by iteration, from the architecture down to the material wiring. Chapter 5 measures the real cost of running the simulation and compares it to the alternatives. Chapter 6 reviews what worked, what the limitations are, and what the future steps are.

Chapter 2

Output Summary

This project delivers four concrete things, each covered in full in Chapter 4:

- A multi-cascade GPU ocean simulation using Tessendorf's FFT method, running as four custom compute shader passes through Unreal's Render Dependency Graph.
- A water colour model whose open-ocean absorption comes from published measurements (Pope and Fry), with the other presets tuned by hand along Jerlov's water types, rather than a colour picked by eye in the material editor.
- `EWaterPreset` / `ApplyWaterPreset` - one function call (or one dropdown and button in the editor) that switches a water body between *Ocean*, *Tropical* and *Murky* looks at runtime, changing the wave shape and the water colour together.
- A comparison of the result against Epic's built-in Water plugin and the shipped-game techniques it is modelled on (Chapter 5).

All of it is open source under the MIT licence, split into a `Source/Ocean` C++ module and a `Shaders/Ocean` HLSL module, so either can be dropped into another Unreal Engine 5 project on its own (the code is on GitHub at `https://github.com/Abhijit2401/OceanFFTWaterSystem`).

Chapter 3

Context

3.1 Wave models in real-time graphics

The cheapest wave model in common use sums a handful of hand-authored sine waves, the *Gerstner wave* approach, popularised for real-time use by Finch (2004). It costs almost nothing per vertex and is fully art-directable, but it cannot reproduce the broad, statistically correct spread of real wind driven water without an impractically large number of hand placed waves, and even so, it starts to look repetitive at scale.

Tessendorf's (2001) method, on the other hand, statistically generates thousands of wave components at once, by treating the ocean surface as a random field with a known frequency spectrum (Section 4.3), then uses the Fast Fourier Transform to turn that frequency domain description into a height field in $O(N^2 \log N)$ time for an $N \times N$ grid which is far cheaper than summing the equivalent number of waves by hand. However, not every shipped game uses it - Guerrilla Games' water for *Horizon Forbidden West* instead plays back offline Houdini simulations baked for real-time use, instanced and blended at runtime (Malan, 2022).

3.2 Water colour in real-time graphics

Most real-time water shaders treat colour as a two-term blend: a reflection term and a see through term produced by refracting the background scene through a handful of hand-tuned constants, as in Crytek's widely used refraction technique for *Crysis* and *Far Cry* (Sousa, 2005). This looks convincing cheaply, but the water's colour is realistically just the background scene tinted by a constant an artist chose and it has no relationship to how real water absorbs and scatters light by wavelength, so it cannot show something as simple as a diver's red equipment fading to grey a few metres down.

Unreal's Single Layer Water shading model takes a more physically grounded approach: it exposes absorption and scattering coefficients, and a scattering direction parameter, as direct shading inputs, and blends a refraction path with a scattering approximation driven by those numbers over the light's path length through the water (Epic Games, 2024). What the model does not supply is what those numbers should actually *be* - that is still a content problem, and it is normally solved the same way Crytek's constants are: an artist picks something that looks right in the viewport. Section 4.7 sources those numbers from measured data instead.

3.3 Water optics

Oceanography already has the measurements a physically based water shader needs. Pope and Fry (1997) measured the absorption spectrum of optically pure water directly, and found that red light (around 650 nm) is absorbed roughly twenty times faster than blue light (around 450 nm) - the actual physical reason clear water reads as blue, rather than the circular answer that "water is blue". Jerlov (1976) classified ocean water into a small number of optical types from decades of shipboard measurements, from Type I (the clearest open ocean) through the more cloudy coastal types which is exactly the axis this project's Ocean/Tropical/Murky presets vary along (Section 4.8).

3.4 Where this project sits

Table 3.1 compares the techniques used in this project with existing alternatives. As stated explicitly in Chapter 5, this does not attempt to outpace the engineering of a shipped AAA title. However, it does show that the *methods* used for both wave generation and water optics follow validated industry practices, rather than the basic approximations usually reached for at this scale.

	Standard real-time approximation	**This project**
Wave shape	1-3 panning normal maps, or a handful of hand-tuned Gerstner sines	Multi-cascade Tessendorf FFT, GPU compute, statistically correct spectrum
Water colour	Single artist-picked gradient + Fresnel	Absorption/scattering coefficients: open-ocean absorption from Pope & Fry, other presets hand-tuned along Jerlov's water types

Table 3.1: This project's physics-driven methods compared to standard, lower-cost rendering approximations.

Chapter 4

Method

This chapter covers how the system was actually built, in the order I built it: the overall architecture first, then the wave simulation piece by piece, then the colour model, then the preset system that ties both together.

4.1 Architecture

The system splits into four pieces (Table 4.1, Figure 4.1): a C++ subsystem that owns state and orchestration, a set of compute shaders that declare the wave maths, the HLSL file that implements it, and a water material that consumes the result.

Location	Responsibility
`Source/Ocean/OceanFFTSubsystem.*`	Owns render targets, CPU-precomputed lookup tables, per-frame dispatch, and the water preset system.
`Source/Ocean/OceanFFTShaders.*`	Declares the four `FGlobalShader` compute shader classes and their parameters.
`Shaders/Ocean/OceanFFT.usf`	The HLSL implementation of all four compute passes.
`Content/Water/Materials/`	The Single Layer Water material: samples the height maps for vertex displacement, and holds the optical parameter nodes from Section 4.8.

Table 4.1: Module responsibilities across the ocean system.

The subsystem is a `UTickableWorldSubsystem` rather than an actor component, because the simulation is a single world-global resource, there is one ocean, not one per actor, so there is no owning actor to find or bind. And all GPU work goes through RDG rather than raw RHI calls, because RDG handles resource lifetime and pass scheduling automatically, which is what let the same compute pattern be reused across three cascades and four passes without hand-written barrier management.

4.2 Iteration 1: rejecting a hand-authored sum of waves

A naive ocean built from W hand-placed sine waves, evaluated per vertex, costs $O(V \cdot W)$ per frame for V vertices, and starts to look repetitive once W is small enough to be affordable. Tessendorf's (2001) insight was to treat the ocean surface statistically instead: rather than choosing W waves by hand, generate a frequency-domain description of every representable wave on an $N \times N$ grid at once, weighted by how likely real wind-driven water is to contain

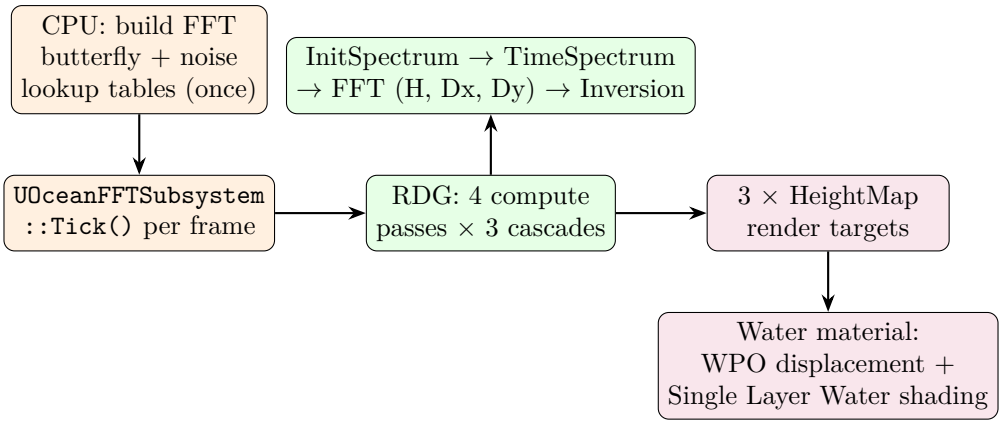


Figure 4.1: Data flow from CPU precompute through GPU simulation to the shaded pixel. The butterfly and noise lookup tables (top-left) are the only CPU-side cost; everything else happens on the GPU every frame via the Render Dependency Graph.

a wave of that frequency and direction (the Phillips spectrum, Section 4.3), then use the FFT to turn that whole field into a spatial height map in $O(N^2 \log N)$ time. That is the method implemented for the rest of this chapter, in `Shaders/Ocean/OceanFFT.usf` and orchestrated in `OceanFFTSubsystem.cpp`. The cost is complexity: four distinct GPU passes and a fair amount of complex-number bookkeeping, all written from scratch rather than pulled from a library.

4.3 Iteration 2: the Phillips spectrum

The Phillips (1958) spectrum gives the expected energy of a wave with wave-vector $\mathbf{k}$, for a wind blowing at speed V in direction $\hat{\mathbf{w}}$:

$$P_h(\mathbf{k}) = A \frac{\exp\left(-\frac{1}{(kL)^2}\right)}{k^4} \left|\hat{\mathbf{k}} \cdot \hat{\mathbf{w}}\right|^2 \exp\left(-k^2 l^2\right) \tag{4.1}$$

where $k = |\mathbf{k}|$, $L = V^2/g$ is the largest wave sustainable by wind speed V, A is an overall amplitude scale, and the last exponential term suppresses wavelengths shorter than l so the finite grid does not alias. `InitSpectrumCS` (Algorithm 1, implemented in Appendix A) evaluates this directly, once per grid cell.

Algorithm 1 `InitSpectrumCS`: initial frequency spectrum, one thread per grid cell

procedure InitSpectrumCS(Pos)
  $\mathbf{k} \leftarrow (\text{Pos} - N/2) \cdot \frac{2\pi}{\text{PatchLength}}$ ▷ centre the wave-vector grid on $\mathbf{k} = 0$
  $\xi \leftarrow$ SampleNoiseTexture(Pos) ▷ four independent $\mathcal{N}(0,1)$ samples, precomputed on CPU
  $P_k \leftarrow A \cdot \text{Phillips}(\mathbf{k}), \quad P_{-k} \leftarrow A \cdot \text{Phillips}(-\mathbf{k})$
  $H_0(\mathbf{k}) \leftarrow \xi_0 \cdot \sqrt{P_k/2}$
  $H_0(-\mathbf{k}) \leftarrow \xi_1 \cdot \sqrt{P_{-k}/2}$
  return $(H_0(\mathbf{k}), H_0(-\mathbf{k}))$ packed into one RGBA texel
end procedure

Both $H_0(\mathbf{k})$ and its partner $H_0(-\mathbf{k})$ are stored together here because the next step, time evolution, needs both to guarantee the final height field comes out real-valued rather than complex.

4.4 Iteration 3: time evolution and choppy displacement

Deep-water waves of wave-number k oscillate at angular frequency $\omega(k) = \sqrt{gk}$. H_0 is evolved forward to time t as

$$H(\mathbf{k}, t) = H_0(\mathbf{k})\, e^{i\omega(k)t} + \overline{H_0(-\mathbf{k})}\, e^{-i\omega(k)t} \tag{4.2}$$

Pairing every frequency with the complex conjugate of its opposite is what keeps the eventual inverse-FFT output real-valued, since a real spatial signal requires exactly this kind of symmetry. On its own, Equation 4.2 only moves the surface vertically. Tessendorf's method also derives a *horizontal* displacement spectrum,

$$\mathbf{D}(\mathbf{k}, t) = i\, \frac{\mathbf{k}}{k}\, H(\mathbf{k}, t) \tag{4.3}$$

which, once inverse-transformed, pulls the mesh horizontally toward wave crests and away from troughs. This is the mechanism that turns plain vertical bobbing into the sharp, pointed crests real ocean water shows. These are usually called *choppy* waves, the FFT counterpart of the horizontal motion in Gerstner waves. $\mathbf{D}$ is split into its x and y components so each can go through its own FFT in the next stage (implementation in Appendix A).

4.5 Iteration 4: the FFT itself

Converting H, D_x and D_y from the frequency domain to the spatial domain uses the Cooley-Tukey radix-2 FFT (Cooley and Tukey, 1965), applied first along rows and then along columns, since a 2D FFT is separable into two passes of 1D FFTs. Each stage combines pairs of elements with the classic butterfly operation (Figure 4.2):

$$X_{\text{top}} = A + W \cdot B, \qquad X_{\text{bottom}} = A - W \cdot B \tag{4.4}$$

where W is the complex twiddle factor for that stage and position. At the first stage, the two input indices must be read from *bit-reversed* positions, a well-known property of the Cooley-Tukey decomposition; at every later stage the indices are read sequentially. Computing bit-reversal and twiddle factors in the shader every frame would be wasted work that never changes, so both are precomputed once on the CPU into a lookup texture and simply read by the shader (Algorithm 2, code in Appendix A).

The twiddle factors use the $e^{-2\pi i}$ sign, which is strictly the forward transform. Because the spectrum is Hermitian-symmetric the output is still real, only mirrored through the origin, and since the $|\hat{\mathbf{k}} \cdot \hat{\mathbf{w}}|^2$ term in the Phillips spectrum weights $\mathbf{k}$ and $-\mathbf{k}$ equally, the mirrored surface is statistically the same ocean.

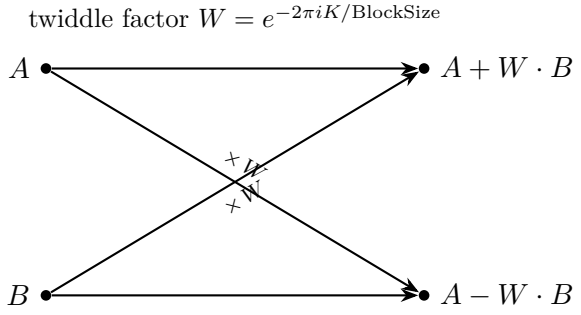


Figure 4.2: One radix-2 Cooley-Tukey butterfly. Two complex inputs A, B combine into two outputs using a single complex multiply and an add/subtract. `FFTButterflyCS` performs exactly this, once per grid cell, per stage, per axis.

Algorithm 2 `FFTButterflyCS`: one butterfly stage, one thread per grid cell

procedure FFTBUTTERFLYCS(Pos, Stage, Direction)
  $c \leftarrow$ Pos.x **if** Direction = horizontal **else** Pos.y
  (IndexA, IndexB, W) $\leftarrow$ BUTTERFLYTEXTURE.LOAD(Stage, c) ▷ precomputed on CPU
  $A \leftarrow$ InputTexture[PosA], $B \leftarrow$ InputTexture[PosB]
  BlockSize $\leftarrow 2^{\text{Stage}+1}$
  if (c mod BlockSize) $<$ BlockSize/2 **then**
    Output[Pos] $\leftarrow A + W \cdot B$
  else
    Output[Pos] $\leftarrow A - W \cdot B$
  end if
end procedure

4.6 Iteration 5: inversion and multi-cascade compositing

The wave-vector grid was centred on $\mathbf{k} = 0$ back in Section 4.3 (shifting the input by half the grid), which means the raw inverse-FFT output comes out mirror-flipped around its centre rather than its corner. The usual fix is a separate `fftshift` pass; instead, `InversionCS` folds the correction into the same pass that packs the output, by multiplying every other pixel by -1 following the parity of $x + y$ (Algorithm 3). This is a standard trick for FFTs whose input has been pre-shifted this way, and it saves one pass compared to the naive approach.

Algorithm 3 `InversionCS`: recentre and pack the final result

procedure INVERSIONCS(Pos)
  $s \leftarrow +1$ **if** (Pos.x + Pos.y) mod $2 = 0$ **else** -1
  Height $\leftarrow s \cdot$ HeightFFT[Pos].$x \cdot$ NormalizationScale
  $D_x \leftarrow s \cdot D_x$FFT[Pos].$x \cdot$ NormalizationScale
  $D_y \leftarrow s \cdot D_y$FFT[Pos].$x \cdot$ NormalizationScale
  Output[Pos] $\leftarrow$ (Height, D_x, D_y, 1) ▷ RGBA render target sampled by the material
end procedure

One FFT grid at one patch size cannot represent both kilometre-scale swells and centimetre-scale ripples without an impractically high resolution. The usual answer, and the one used here, is cascades: this project runs the whole four-pass pipeline three times per frame at three patch lengths: 20 m, 5 m and 1.2 m, and sums their contributions in the material, each weighted separately for height, normal and choppiness (tunable live through the `Ocean.HeightWeight*` / `Ocean.NormalWeight*` / `Ocean.Choppiness*` console variables). Cascade 0, the largest patch, gives the broad swell shape; cascade 2, the smallest, supplies the fine ripple detail that keeps close-up water from looking glassy.

4.7 Iteration 6: a physically grounded water colour

Water's colour comes from light being absorbed and scattered as it travels through a volume, not from a property of its surface. The simplest model of this, Beer-Lambert attenuation, says light of a given wavelength travelling a distance d through a medium with absorption coefficient a and scattering coefficient b is attenuated as

$$L(d) = L_0 \, e^{-(a+b)\,d} \tag{4.5}$$

Unreal's Single Layer Water shading model exposes exactly these quantities as material inputs, `AbsorptionCoefficients` and `ScatteringCoefficients` (both per-channel RGB, since attenuation depends on wavelength), and a `PhaseG` term for how forward- or back-scattered the light

is and blends a refraction path with a scattering approximation driven by them (Epic Games, 2024). The engine handles the shading correctly; the open question is what numbers to put in those three inputs.

Instead of picking a colour that looks like clear ocean water, the open-ocean preset's absorption coefficients are Pope and Fry's (1997) measured pure-water absorption spectrum, sampled at red/green/blue wavelengths and converted from their published m^{-1} units into Unreal's centimetre-based units. In the material this is a `VectorParameter` node named `AbsorptionCoefficients` set to (0.0034, 0.0006, 0.00015) and a second node, `ScatteringCoefficients`, set to (0.0003, 0.0004, 0.0005), wired directly into the matching pins on the material's `SingleLayerWaterMateria` node (Figure 4.3). Pope and Fry only measured absorption, so the scattering values are not from their data; they are set to roughly the magnitude Jerlov reports for clear oceanic water rather than taken from one measured spectrum. Red light attenuates roughly twenty times faster than blue in this data. This is why deep clear water reads as blue in this model. Table 4.2 lists the equivalent values for all three presets.

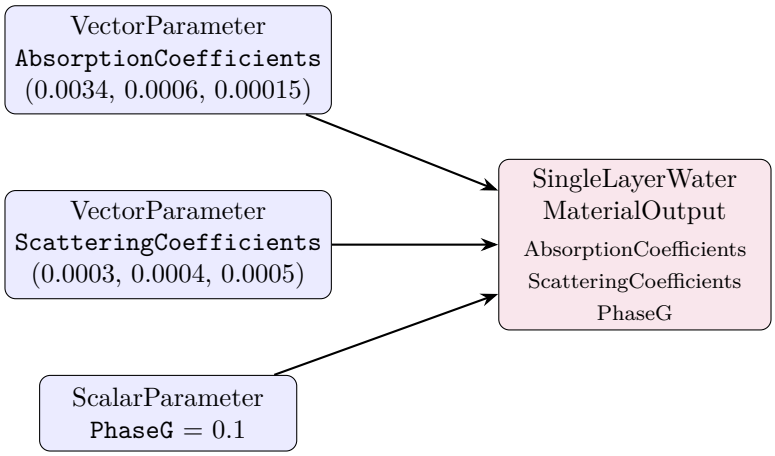


Figure 4.3: The base water-colour wiring, isolated from the rest of the material graph: three parameter nodes feeding directly into the Single Layer Water output node. Section 4.8 extends this with a blend chain in front of each pin.

4.8 Iteration 7: Jerlov presets and the biome blend

Open ocean is only one point on Jerlov's scale. Coastal and tropical-lagoon water contains far more dissolved organic matter and suspended particles: coloured dissolved organic matter absorbs blue light more aggressively than open ocean does, so green penetrates deepest instead of blue. This is the reason tropical shallows look turquoise rather than deep blue. Turbid river or lake water goes further still: dissolved organic matter and sediment absorb blue light strongly, which is the main reason murky water looks brown. Table 4.2 and Figure 4.4 give the numbers for all three presets as implemented. Only the Ocean absorption values are measured data; the Tropical and Murky values were tuned by hand to sit along Jerlov's coastal and turbid water types.

The material also supports a continuous spatial blend between all three presets across world-space position, so one body of water can transition from a river mouth to open ocean without a hard seam (Figure 4.5). This is a two-stage linear interpolation chain in the material graph: Murky blends into Tropical over one world-space range, then that result blends into Ocean over a second, further-out range. The same two-stage chain runs three times in parallel, once each for absorption, scattering and phase-g, all driven by the same two blend weights.

The material's full graph runs to roughly 90 expressions once wave sampling, the Fresnel/refraction

	Absorption ($\times 10^{-3}\,\mathrm{cm}^{-1}$)			Scattering ($\times 10^{-3}\,\mathrm{cm}^{-1}$)		
Preset	R	G	B	R	G	B
Ocean (Jerlov I)	3.4	0.6	0.15	0.3	0.4	0.5
Tropical (Jerlov III)	4.0	1.2	2.5	2.0	2.5	3.0
Murky (turbid)	6.0	5.0	8.0	8.0	6.0	4.0

Table 4.2: Absorption and scattering coefficients per preset, from `GetPresetSettings`. Ocean absorption is from Pope and Fry (1997); everything else is hand-tuned. The Murky preset's scattering is weighted towards red to push its colour further towards brown, which is a look choice rather than a measured property.

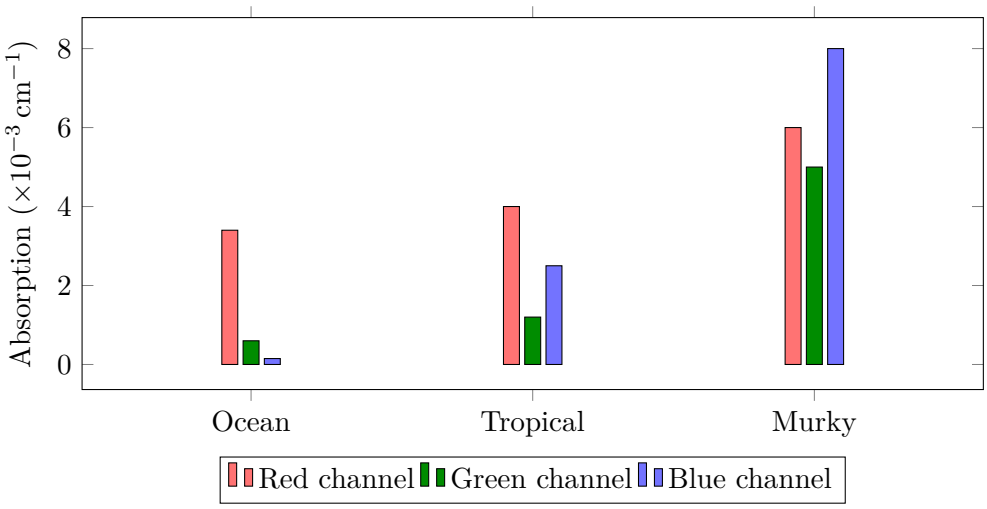


Figure 4.4: Per-channel absorption coefficients across the three presets. Ocean shows the steep red≫green>blue drop-off of clear water; Murky is flatter and elevated across all channels, matching turbid water's much shorter visibility depth.

logic, normal reconstruction, and foam/shoreline blending are included, and Unreal's own graph editor lays that out as a dense tangle of wires. Figure 4.5 plots only the blend-weight ramp; the individual parameter and Lerp nodes that consume it are not reproduced here, since they add wiring detail without changing what the ramp itself is doing.

4.9 Iteration 8: from hand-wiring to a single function call

Building the material graph is a one-time editor task, but changing presets still meant adjusting nodes by hand, which ruled out runtime changes such as a lake turning murky in-game.

Instead of altering the graph, the fix drives the parameters it already exposes (`AbsorptionCoefficients`, `ScatteringCoefficients`, `PhaseG`) through the subsystem's dynamic material instance (`ApplyWaterPreset`, Appendix A). The presets live in a switch statement over a plain struct, so adding one takes a single `case`. It is exposed both as a `CallInEditor` button and as a `BlueprintCallable` function, so presets can now be switched at runtime as well as in the editor.

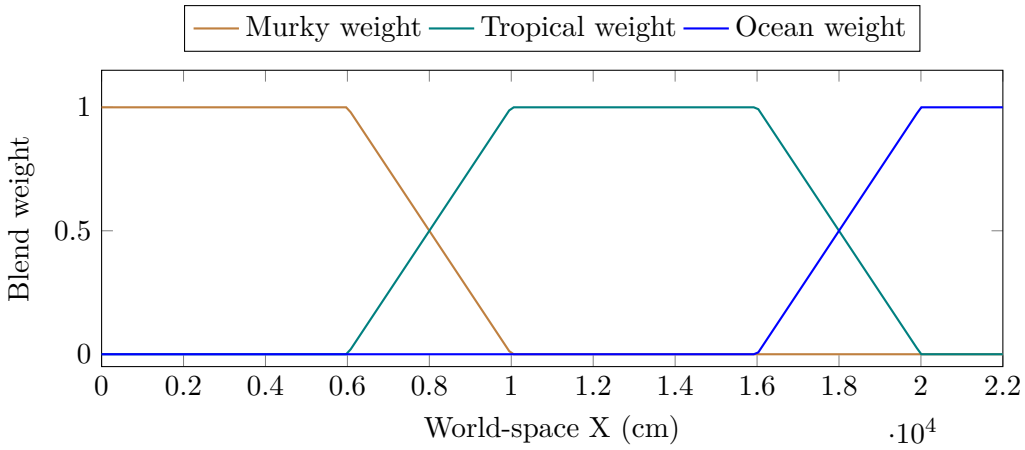


Figure 4.5: The two-stage blend ramp: Murky $\rightarrow$ Tropical over $x \in [6000, 10000]$ cm, then Tropical $\rightarrow$ Ocean over $x \in [16000, 20000]$ cm. Each preset's optical coefficients are linearly interpolated by these weights before reaching the Single Layer Water output node.

Chapter 5

Results

5.1 Cost of the simulation

With `GridSize` = 256 (so $\log_2 N = 8$), each of the three fields (height, D_x, D_y) needs $2 \log_2 N = 16$ butterfly dispatches (8 horizontal, 8 vertical), giving $3 \times 16 = 48$ butterfly dispatches, plus one `TimeSpectrumCS` and one `InversionCS` dispatch, making 50 compute dispatches per cascade per frame in steady state (`InitSpectrumCS` only runs again if wind parameters change). At three cascades, that is **150** compute dispatches through RDG every frame, each covering a 256×256 thread grid. This is a directly computed figure, not a measured one, since the project has no profiling data beyond the correctness checks described below, but it is also the main lever for a performance budget: halving `GridSize` to 128 roughly halves the dispatch count while only dropping $\log_2 N$ by one stage.

5.2 Correctness testing

There is no ground-truth reference ocean to diff pixels against, so testing here means checking the simulation is producing sane, non-degenerate output rather than measuring frame time. `Ocean.DebugLog 1` periodically logs cascade 0's centre pixel (height, D_x and D_y), so a NaN, a flat zero, or a value that stops changing over time is visible immediately in the log rather than only showing up as a subtle visual artefact. In practice this caught two classes of problem during development: a wrongly bound render target (the debug log stayed at exactly zero), and a forgotten wind-parameter reset on preset switch (the log kept producing old-preset values for one frame after `ApplyWaterPreset` ran). Both were visual/behavioural bugs that would have been much slower to track down from the rendered image alone.

5.3 Comparison against the alternatives

Table 5.1 compares this project against Epic's built-in Water plugin, the typical simplified approach at student/indie scale, and the two shipped-game references used throughout this report.

5.4 Discussion

This project's ocean is not going to beat *Horizon Forbidden West*'s water on raw visual fidelity, and it should not be expected to, since that would be comparing a few weeks of individual work against a shipped AAA title built by a dedicated rendering team over years, with a full PS5 frame budget and proprietary multi-scattering and foam simulation this project does not attempt

	This project	**UE5 Water plugin**	**Typical indie shader**	**Sea of Thieves / Horizon FW**
Wave method	Custom multi-cascade FFT (compute)	Built-in Gerstner actors	Panning normal maps	FFT (SoT); baked offline sims played back at runtime (HFW)
Colour method	Measured absorption (Ocean preset), hand-tuned elsewhere	Artist-tunable, not measurement-driven	Hand-picked gradient	Proprietary, art-directed
Runtime cost	150 compute dispatches/frame at 256^2 (computed)	Low (simplified per-instance waves)	Very low	High, budget-tuned per title
Extensibility	Data-driven presets, open source	Engine-locked, closed source	Ad hoc	Studio-internal

Table 5.1: Comparison across wave method, colour method, cost and extensibility.

to do. Instead, the wave method I am using here is the same published FFT technique *Sea of Thieves* shipped with, and the colour inputs start from measured data, rather than the simplified approximation (a few sine waves, a picked gradient) that would normally be reached for at this scale. This is why I avoided the Epic Water plugin. Using a finished system would have shown I can use the editor, not that I can build the technique the editor wraps.

Chapter 6

Conclusions and Discussion

6.1 Objectives review

The goal was to build both halves of realistic water (motion and colour) from first principles on Unreal's raw rendering primitives, without leaning on the engine's own water system.

- **Wave simulation: successful.** A multi-cascade GPU compute-shader ocean using Tessendorf's FFT method is implemented and running (Chapter 4).
- **Water colour: successful.** The open-ocean absorption comes from published measurements and the other presets are tuned along Jerlov's water types, rather than an authored gradient (Section 4.7).
- **Reusable preset system: successful.** `ApplyWaterPreset` switches both systems together behind one call, with no material-editor work needed (Section 4.9).

6.2 Limitations

- All three cascades run at full resolution every frame regardless of camera distance; there is no LOD or update-rate throttling for distant water.
- Grid size and cascade patch lengths are fixed at subsystem initialisation; changing them at runtime means re-running `Initialize`, not a hot-swappable parameter.
- `ApplyWaterPreset` sets one uniform look for a whole water body. The continuous spatial blend from Section 4.8 exists in the material but is not yet exposed through the preset API.
- Foam, spray and shoreline interaction are handled by separate material layers rather than being derived from the FFT's own steepness data, which is the more physically grounded approach some of the cited references use.
- There is no measured frame-time or GPU profiling data in this report, only the computed dispatch-count figure in Section 5. A real performance budget would need that measured, not just estimated.

6.3 Future work

The most useful next step is exposing the spatial biome blend through `ApplyWaterPreset` so a single water body can transition types without hand-authoring the blend ranges per level. After that, distance-based LOD for the cascades would recover GPU time on levels where most of the

ocean is far from the camera, and deriving foam directly from the FFT's Jacobian would remove the current dependency on separately authored foam layers.

6.4 Licensing and reuse

The code (C++ and HLSL) is released under the MIT licence, so it can be dropped into another Unreal Engine 5 project as long as the original copyright notice is kept. Writing it as a self-contained module rather than a level-specific hack was always meant to make it something other people could reuse.

6.5 Closing

This report set out to answer one question: could both halves of realistic water be built from first principles on Unreal Engine's raw rendering primitives, instead of leaning on the engine's own pre-built water system? The answer implemented here is a multi-cascade GPU compute-shader ocean using Tessendorf's FFT method, and a water colour model that starts from real published measurements rather than an authored gradient, unified behind one reusable, runtime-callable preset switch.

References

1. Ang, N., Catling, A., Cifariello Ciardi, F. and Kozin, V. (2018) 'The Technical Art of *Sea of Thieves*', *Proceedings of SIGGRAPH '18 Talks*. Vancouver: ACM.

2. Cooley, J. W. and Tukey, J. W. (1965) 'An Algorithm for the Machine Calculation of Complex Fourier Series', *Mathematics of Computation*, 19(90), pp. 297-301.

3. Epic Games (2024) 'Single Layer Water Shading Model in Unreal Engine', *Unreal Engine 5 Documentation*.

4. Finch, M. (2004) 'Effective Water Simulation from Physical Models', in Fernando, R. (ed.) *GPU Gems*. Boston: Addison-Wesley, ch. 1.

5. Jerlov, N. G. (1976) *Marine Optics*. 2nd edn. Amsterdam: Elsevier.

6. Malan, H. (2022) 'Rendering Water in Horizon Forbidden West', *SIGGRAPH 2022 Advances in Real-Time Rendering in Games Course*. Guerrilla Games / ACM.

7. Phillips, O. M. (1958) 'The Equilibrium Range in the Spectrum of Wind-Generated Waves', *Journal of Fluid Mechanics*, 4(4), pp. 426-434.

8. Pope, R. M. and Fry, E. S. (1997) 'Absorption Spectrum (380-700 nm) of Pure Water. II. Integrating Cavity Measurements', *Applied Optics*, 36(33), pp. 8710-8723.

9. Sousa, T. (2005) 'Generic Refraction Simulation', in Pharr, M. (ed.) *GPU Gems 2*. Boston: Addison-Wesley, ch. 19.

10. Tessendorf, J. (2001) 'Simulating Ocean Water', *SIGGRAPH 2001 Course Notes: Simulating Nature: Realistic and Interactive Techniques*. Los Angeles: ACM.

Appendix A

Code Listings

The listings below are the actual implementation behind the algorithms and equations in Chapter 4, pulled out of the main text so that chapter reads as an explanation rather than a code dump. The complete, current source lives in `Source/Ocean/OceanFFTSubsystem.h/.cpp`, `Source/Ocean/OceanFFTShaders.h/.cpp`, and `Shaders/Ocean/OceanFFT.usf`.

A.1 The Phillips spectrum (Section 4.3)

```
float PhillipsSpectrum(float2 K, float2 WindDir, float WindSpeed, float
    SuppressSmallWaves)
{
    float KLen = length(K);
    if (KLen < 0.00001f)
    {
        return 0.0f;
    }

    float L = (WindSpeed * WindSpeed) / GRAVITY; //largest wave for this wind speed
    float KLen2 = KLen * KLen;
    float KDotW = dot(normalize(K), WindDir);

    float Phillips = exp(-1.0f / (KLen2 * L * L)) / (KLen2 * KLen2);
    Phillips *= KDotW * KDotW;

    //kill waves smaller than SuppressSmallWaves so the grid doesnt alias
    float SmallSuppressor = exp(-KLen2 * SuppressSmallWaves * SuppressSmallWaves);
    return Phillips * SmallSuppressor;
}
```

Listing A.1: The Phillips spectrum, Equation 4.1. From `OceanFFT.usf`.

A.2 Time evolution and choppy displacement (Section 4.4)

```
float2 NCoord = float2((float)Pos.x - (float)GridSize * 0.5f, (float)Pos.y - (float)
    GridSize * 0.5f);
float2 K = NCoord * (PI2 / PatchLength);
float KLen = max(length(K), 0.00001f);
float Omega = sqrt(GRAVITY * KLen);

float4 H0 = H0Texture.Load(int3(Pos, 0));
float2 H0K = H0.xy;
float2 H0MinusKConj = float2(H0.z, -H0.w);

float CosOt = cos(Omega * Time);
float SinOt = sin(Omega * Time);
float2 EulerPos = float2(CosOt, SinOt);
float2 EulerNeg = float2(CosOt, -SinOt);

```

```
float2 Hkt = ComplexMul(H0K, EulerPos) + ComplexMul(H0MinusKConj, EulerNeg);
HktOutput[Pos] = Hkt;

//choppy displacement: D(k,t) = i * (k/|k|) * H(k,t), from Tessendorf 2001
float2 KDir = K / KLen;
float2 IKx = float2(0.0f, KDir.x);
float2 IKy = float2(0.0f, KDir.y);
DxOutput[Pos] = ComplexMul(IKx, Hkt);
DyOutput[Pos] = ComplexMul(IKy, Hkt);
```

Listing A.2: Dispersion relation and choppy displacement spectra, Equations 4.2 and 4.3. From `OceanFFT.usf`.

A.3 The FFT butterfly (Section 4, Iteration 4)

```
for (int32 Stage = 0; Stage < LogGridSize; Stage++)
{
    int32 BlockSize = 1 << (Stage + 1);
    int32 HalfBlock = BlockSize / 2;

    for (int32 X = 0; X < GridSize; X++)
    {
        int32 BlockStart = (X / BlockSize) * BlockSize;
        int32 PosInBlock = X % BlockSize;
        int32 K = PosInBlock % HalfBlock;

        float Angle = -PI * 2.0f * (float)K / (float)BlockSize;
        float TwiddleRe = FMath::Cos(Angle);
        float TwiddleIm = FMath::Sin(Angle);

        int32 IndexA, IndexB;
        if (Stage == 0)
        {
            IndexA = (int32)BitReverse(BlockStart + K, LogGridSize);
            IndexB = (int32)BitReverse(BlockStart + K + HalfBlock, LogGridSize);
        }
        else
        {
            IndexA = BlockStart + K;
            IndexB = BlockStart + K + HalfBlock;
        }

        ButterflyCPUData[X * LogGridSize + Stage] = FVector4f((float)IndexA, (float)
            IndexB, TwiddleRe, TwiddleIm);
    }
}
```

Listing A.3: Precomputing bit-reversal indices and twiddle factors once on the CPU, so the shader only ever does a texture lookup. From `OceanFFTSubsystem.cpp`.

```
uint AxisCoord = (Direction == 0) ? Pos.x : Pos.y;
float4 Butterfly = ButterflyTexture.Load(int3(Stage, AxisCoord, 0));
uint IndexA = (uint)Butterfly.x;
uint IndexB = (uint)Butterfly.y;
float2 Twiddle = Butterfly.zw;

uint2 PosA = (Direction == 0) ? uint2(IndexA, Pos.y) : uint2(Pos.x, IndexA);
uint2 PosB = (Direction == 0) ? uint2(IndexB, Pos.y) : uint2(Pos.x, IndexB);

float2 A = InputTexture.Load(int3(PosA, 0));
float2 B = InputTexture.Load(int3(PosB, 0));
float2 TB = ComplexMul(Twiddle, B);

uint BlockSize = 1u << (Stage + 1u);
uint HalfBlock = BlockSize >> 1u;
bool bTop = (AxisCoord % BlockSize) < HalfBlock;

OutputTexture[Pos] = bTop ? (A + TB) : (A - TB);
```

Listing A.4: The butterfly shader: every stage is the same tiny kernel, driven entirely by the precomputed lookup texture. From `OceanFFT.usf`.

A.4 Multi-cascade orchestration (Section 4.6)

```
auto RunFFT = [&](FRDGTextureRef PingA, FRDGTextureRef PingB, const TCHAR* EventName) ->
    FRDGTextureRef
{
    FRDGTextureRef CurrentInput = PingA;
    FRDGTextureRef CurrentOutput = PingB;
    for (int32 Stage = 0; Stage < LocalLogGridSize; Stage++)
    {
        // ... dispatch FFTButterflyCS, Direction = 0 (horizontal) ...
        ::Swap(CurrentInput, CurrentOutput);
    }
    for (int32 Stage = 0; Stage < LocalLogGridSize; Stage++)
    {
        // ... dispatch FFTButterflyCS, Direction = 1 (vertical) ...
        ::Swap(CurrentInput, CurrentOutput);
    }
    return CurrentInput;
};

FRDGTextureRef HeightResult = RunFFT(HeightPingA, HeightPingB, TEXT("OceanFFTHeight"));
FRDGTextureRef DxResult = RunFFT(DxPingA, DxPingB, TEXT("OceanFFTDx"));
FRDGTextureRef DyResult = RunFFT(DyPingA, DyPingB, TEXT("OceanFFTDy"));
```

Listing A.5: Per-cascade FFT orchestration inside the RDG render command: the same four-pass pipeline, run once per cascade, per frame. Abbreviated from `OceanFFTSubsystem.cpp`.

A.5 The preset system (Section 4.9)

```
void UOceanFFTSubsystem::ApplyWaterPreset(EWaterPreset Preset)
{
    ActiveWaterPreset = Preset;
    const FWaterPresetSettings Settings = GetPresetSettings(Preset);

    //feeds straight into the Phillips spectrum / dispersion relation next Tick, so
    //cascade 0 re-initialises with the new wind/amplitude
    WindSpeed = Settings.WindSpeed;
    WindDirection = Settings.WindDirection;
    Amplitude = Settings.Amplitude;
    SuppressSmallWaves = Settings.SuppressSmallWaves;
    for (FOceanCascadeRuntime& Cascade : Cascades)
    {
        Cascade.bSpectrumInitialized = false;
    }

    //pushed straight to the SLW material params already wired up in the water
    //material's graph (VectorParameter/ScalarParameter nodes -> SLW output node)
    if (UMaterialInstanceDynamic* WaterMID = CachedWaterMID.Get())
    {
        WaterMID->SetVectorParameterValue(TEXT("AbsorptionCoefficients"), Settings.
            AbsorptionCoefficients);
        WaterMID->SetVectorParameterValue(TEXT("ScatteringCoefficients"), Settings.
            ScatteringCoefficients);
        WaterMID->SetScalarParameterValue(TEXT("PhaseG"), Settings.PhaseG);
    }
}
```

Listing A.6: The full water-type switch: one function call updates both the wave simulation and the water material. From `OceanFFTSubsystem.cpp`.

```
case EWaterPreset::Tropical:
    //Shallow coastal/lagoon water (Jerlov Type III). CDOM eats blue way harder than
    //open ocean and green goes deepest, that's the real reason this reads turquoise
    //not deep blue. Calmer, shorter waves than the ocean preset too.
    Settings.WindSpeed = 250.0f;
    Settings.WindDirection = FVector2D(1.0, 0.3);
    Settings.Amplitude = 2.5f;
    Settings.SuppressSmallWaves = 0.4f;
    Settings.AbsorptionCoefficients = FLinearColor(0.004f, 0.0012f, 0.0025f);
    Settings.ScatteringCoefficients = FLinearColor(0.002f, 0.0025f, 0.003f);
    Settings.PhaseG = 0.3f;
    break;
```

Listing A.7: One preset entry from the data-driven preset table. Adding a fourth preset means adding one more `case`, nothing else. From `OceanFFTSubsystem.cpp`.

Declaration of AI Use

I used AI such as Gemini and Visual Studio Code's in-built predictive capabilities while building this project and refining this report, and I want to be specific about where rather than leave it as a vague disclaimer:

- **Research.** I used AI to find and fact check a few sources and papers, and to summarise them into a form I could read quickly. I did not use AI to generate any of the technical content in this report, or to make any engineering decisions about what to implement or how.
- **Writing.** This report itself was drafted with AI assistance using a combination of Gemini (to mimic my own University Dissertation layout to form a template) and Visual Studio Code's predictive capabilities. I used it to help with grammar, phrasing, and clarity, but all technical content, explanations, and conclusions were written by me. I also used AI to help format the LaTeX code and ensure that the document compiled correctly.

None of the engineering decisions such as what to build, why to build it that way, and whether it actually works were made by AI.